

Approaching Entropy Limits with Learned Lossless Compression

Grant Proposal

Ruchir Kafle

Massachusetts Academy of Math and Science at the Worcester Polytechnic Institute

85 Prescott Street, Worcester, Massachusetts

Abstract (RQ) or Executive Summary (Eng)

Data is the most important currency in the modern world. Increasingly more data is being stored by the day, however, data storage is costly, both monetarily and environmentally. To combat this problem, methods of compressing data into smaller spaces were developed, simply called data compressors. While these methods made great leaps in storing more data, many compressors eventually reach an incomplete upper limit to how much a given piece of data can be compressed. This project aimed to prove the ability of data compressors to improve furthermore, and reach the end goal that is entropy, the minimum size some data can losslessly be represented within, specifically regarding text data. To do this, a relatively novel idea was implemented, that being a hybrid data compressor, combining two large subsections of data science, learned or neural compression, where neural networks are used, along with traditional compression, where they are not. Along with that, a large language model (LLM) was used during learned compression, differing from typical neural compressors based on recurrent neural networks (RNNs) or transformers. The implemented compressor was then tested against a variety of text datasets, with resulting compressed sizes being compared to a calculated theoretical entropy. This comparison was used to judge the effectivity of the created compressor, and judge whether entropy can be reached with this approach. The conclusion of this study may serve significant to how data centers and other institutions or consumers store data, helping determine their rate of expansion.

Keywords: Large Language Models, Data Compression, Neural Compression, Learned Compression, Entropy Limits, Hybrid Compressor

Approaching Entropy Limits with Learned Lossless Compression

Can a data compressor condense all multimedia data down to their entropy limit? It is hypothesized that a hybrid model, combining a neural network with a greater memory and semantic understanding of text, along with traditional models, can indeed approach the entropy limit.

A data compression algorithm is a program that attempts to find patterns within input information and extract them, creating the effect of the same information taking up less space. Data compression algorithms are essential, being used everywhere, so users and companies alike can store more information without having to get more expensive hardware. These algorithms sprouted in response to Shannon's Information Theory, a book which proves that all data has a theoretical minimum amount of space in which it can take up (Shannon, 1948). This minimum amount of space is called the entropy rate, specifically being the minimum average number of bits that each unit in a sequence will take up. Data compression algorithms, such as JPEG, Huffman coding, and LZMA, all specializing in compressing different forms of media, such as images, text, etc., and reach this discussed entropy rate.

Reaching the entropy limit, or minimum entropy rate, is no menial task, hence why it's often only seen theoretically and not experimentally. For decades, universal data compression algorithms, that is data compression algorithms that work on all sorts of media, have attempted this in multiple ways. Often, they scan through a file and try to find recurring symbols, which are counted up and then fed into an entropy coding block. An entropy coding block operates by seeing how often certain symbols appear in a sequence, and based on how common a character is, it is assigned a correspondingly large bit sequence (Moffat, 2019). If a sequence of 10 characters is mostly composed of the letter A, containing six of them, data compression algorithms will see that and give it the smallest bit sequence. If the letter Z only appears in that same sequence a single time, it will be given a larger bit sequence. If the process was inverted, giving larger sequences to common characters and giving shorter sequences to uncommon characters, the result would be a compression larger than what was otherwise possible. Two

examples of entropy coding are Huffman coding and arithmetic coding, both of which common in data compression, and the latter being implemented in this project. These data compression algorithms are known as traditional compressors (Sun et al., 2025).

While traditional algorithms do work, they aren't perfect. Traditional algorithms can compress all forms of media close to their limits, but they only ever get close. The issue with them is that their ability to find complex patterns within the data is lacking, preventing them from compressing further. Attempting to solve this issue, Schmidhuber and Heil (1996) proposed the use of neural networks in data compression, due to neural networks' abilities to recognize patterns. Neural networks are the intelligent systems behind the artificial intelligence (AI) programs often used today, such as ChatGPT and Gemini. The use of neural networks within data compression is called neural compression (Yang, 2023), or as it will be referred to here, learned compression. In their proposal, Schmidhuber and Heil (1996) suggested a compressor made up of a probability predictor block, leading into an encoder block, which isn't too dissimilar from traditional compressors. The only difference between the two models is that the probability predictor block in learned compression is composed of a neural network, with the encoding block staying the entropy coding block staying the same. These sections come together as the neural network block will predict the probability of what character is going to come next in a sequence, called the probability distribution, which will then be fed into the encoder block to calculate bit sequence lengths per character.

Learned compression has advanced since. From simple feedforward networks like in Schmidhuber and Heil's work, other learned compressors were created, like DeepZip, from Goyal et al. (2018). DeepZip expands on previous work through the use of a recurrent neural network (RNN), which allows the algorithm to better remember previous terms in a sequence during prediction, giving better probability distributions. From there, the authors created DZip in 2020 to further improve the speed of the RNN used. Mao et al. (2022) further went on, using a Transformer in their model to weigh semantic

value, providing better probabilities. The transformer model has a system called attention, which is simply an algorithm that weighs the importance of each part of a sequence, seeing what has the most value. Attention is used to see what part of a sequence should be given the most attention by the model when determining an output.

Despite these advances, learned compression has many problems as of now. While learned compression models are approaching the entropy limit, they are still quite a bit off. These models have two main issues: runtime and approaching entropy. Due to the use of neural networks, which are large and expensive algorithms, they typically run many orders of magnitude slower than typical processors. Against enwik9, a dataset with a total size of 1 gigabyte, traditional compressors typically took a few minutes, while learned compressors typically took hours or even days (Mahoney, 2025). Additionally, due to the use of neural networks, specialized hardware is required to run the high computational demands of the compressor. As such, neural compression is not a widely accessible field as of now.

To further compress files, this paper will focus on two implementations: creating a hybrid model, and a prediction block using a large language model. A bias will be placed on compressing text files due to the use of semantic understanding in a large language model. A hybrid model will be created, meaning the use of traditional compressors along with neural, in order to create a failsafe along with decreasing compression times.

Section II: Specific Aims

This proposal's objective is to prove that a data compression algorithm can indeed reach the entropy limit for any piece of data. More specifically, it aims to implement a hybrid data compression algorithm using an LLM for greater semantic understanding in text. The LLM will likely be from ChatGPT-2 or LLaMA-2 (Touvron et al., 2023) to prevent the need to train an LLM myself.

Our long-term goal is to procedurally build a hybrid data compression model, capable of compressing data further than previous models. The central hypothesis of this proposal is that a hybrid model, combining a neural network with a greater memory and semantic understanding of text, along with traditional models, can indeed reach the entropy limit. The rationale is that neural compressors have been advancing at rapid paces, getting closer and closer to the entropy limit, however remaining just a fraction of the size off (Mahoney, 2025). There are still so many places for learned compressors to grow, so with the implementation of a large feature, it should be able to affect the compression ratio significantly enough to bring the compression size right to the limit. The work we propose here will prove once and for all that the theoretical can become practical, with the final model producing compressed files within a few percent of the theoretical entropy limit for any dataset which it is passed in. This will include datasets such as enwik9 and dickens. Along with being a proof, this work may help datacenters store more data than previously, and maybe even on a smaller scale with consumer devices.

Specific Aim 1: Utilize a large language model to add semantic understanding to text data compression.

Specific Aim 2: Add traditional data compression algorithms to the encoder block.

Specific Aim 3: Prove the model is lossless and compare the final entropy rate with the theoretical limit as calculated with the appropriate formula from Shannon (1948).

The expected outcome of this work is a hybrid data compression model incorporating an LLM along with various traditional compressors like LZMA77 that can indeed reach the entropy limit.

Section III: Project Goals and Methodology

Relevance/Significance

Data is the most important thing in our world today. We use data on a daily basis to do all sorts of things, like learn new things, remember past experiences, do our jobs, etc. And the amount of data being stored globally is only increasing (Sun et al., 2025). This places a burden on companies and users that want to store more data themselves. As in order to do so, the cost can at times be quite exorbitant. This project will help reduce some of the strain that may occur when wanting to store more data, preventing one from having to buy more storage. Instead, a user will simply be able to compress their current data, freeing up storage space and allowing them to store more. If they ever wish to access and use that data, they can simply decompress it with the same algorithm that was used in compression.

Innovation

It's very common to see LSTMs being used in learned compression; however, this model differs as it will be using an LLM. While LLMs have been done a few times, like in Valmeekam et al. (2023), along with Transformers, this model aims to obtain much better predictions than were proved to be possible in previous works. In addition to that, this project will incorporate traditional compressors with the neural compressor to achieve a final compressor with all the pros of each.

Methodology

Specific Aim #1: Utilize a large language model to add semantic understanding to text data compression.

The objective of this aim is to add semantic understanding to our model when compressing text data. Our approach is to utilize pre-trained large language models for this project, such as LLaMA-2.0 (Touvron et al., 2023). We will feed the large language model on the text files we are given to compress and will then extract the probability distributions from the model to put through the arithmetic encoder.

Our rationale for using a large language model is that it has greater semantic understanding when it comes to language than other models, theoretically enabling it to compress language far better than previously possible (Valmeekam et al., 2023).

Justification and Feasibility. Through the utilization of a large language model, the neural network implemented in the probability block will have a greater understanding over text structure than other machine learning algorithms. Large language models are comprised of a transformer, allowing them to utilize attention, and as previously discussed, allows greater understanding over language.

Summary of Preliminary Data. It has been proved that learned compressors are much better at compression than regular compressors are. Bellard (n.d.) ran a few tests and found the following, with exception to the final column, percentage ratio, which was calculated by this paper:

Program	Compr. size (bytes)	Ratio (bpb)	Program size (zip, bytes)	Total (bytes)	%
gzip	322,591,995	2.58	38,801	322,630,796	32.2%
xz	197,331,816	1.58	36,752	197,368,568	19.7%
CMIX (v19)	111,470,932	0.892	223,485	111,694,417	11.1%
NNCP (2023-10-21)	106,632,363	0.853	628,955	107,261,318	10.6%

According to the data found by Bellard (n.d.), neural compressors, such as NNCP and CMIX, are able to compress files to much smaller sizes than traditional compressors such as xz and gzip. This proves their potential in compression and optimizing storage.

Additionally, this study ran various learned and traditional compressors against a smaller, 10-megabyte dataset, dickens, as provided in Mahoney (n.d.):

Compressor	Default	CMix	TRACE	7z, bz2, xz	zst	zpaq, zlib, zip, gz	lzo, rar
------------	---------	------	-------	-------------	-----	---------------------	----------

Storage (MB)	10	1.8	2.6	2.8	3.7	3.9	6.2
Ratio	1	5.56	3.85	3.57	2.70	2.56	1.61
%	100%	18%	26%	28%	37%	39%	62%

As seen in the table, the neural compressors, CMix (Byron, n.d.) and TRACE (Mao et al., 2022), were yet again much more efficient than the traditional compressors in compressing data. Between the two tables, a noticeable difference in compression ratios can be noticed, where each compressor on average compresses further on the larger enwik9 than dickens. This shows the immense potential of compression in large data, where it is most needed.

Valmeekam et al. (2023) goes on to prove that large language models are a significant competitor in lossless learned compression. They show that large language models are able to produce very accurate probability distributions for text data, enabling greater compressions. This gives us the ability to compress closer to the entropy limit than seems possible in the tables above.

Expected Outcomes. The overall outcome of this aim is to better compress text data with learned compression, proving that large language models can successfully be used in compression. This knowledge will be used for compression of large amounts of data where text is the primarily stored data type.

Potential Pitfalls and Alternative Strategies. We expect that the process of installing a large language model into the project will not be a straightforward process. There will likely be many hard to navigate and understand aspects, along with some outdated hardware that will lead to significant issues. Some ways we can work around these issues are by looking for the newest large language models to run prior to finalizing one and in the case, we do find issues; we can scrap that model in preference for one that works.

Specific Aim #2: Add traditional data compression algorithms to the encoder block.

The objective of this aim is to create another block of the compressor where traditional compressors will further compress a file after the neural compressor. Once the given information has gone through the arithmetic encoder block to be compressed, it will be output and immediately put right into the traditional encoding block, where the file will be checked again for compressible data. Our rationale for doing so is that any compressible data that is missed or done inefficiently by the neural compressor will be able to be run through again by traditional compressors. This will act as a failsafe to ensure that all data is compressed as much as possible. Multiple traditional compression algorithms will be implemented, such as LZ77.

Justification and Feasibility. In Fu et al. (2019), it is proven that a neural compressor used in conjunction with traditional compressors can compress a file a smaller than either of the components separately. This study, however, does compress images; however, our project predicts that the same concepts used in these previous works can be applied to text data as well.

Summary of Preliminary Data. Fu et al. (2019) proves that hybrid compression does work for images, however, does only provide small benefits compared to other compressors. As said, images were experimented on for this paper, which is why it can't quite be predicted how the results will vary for text. We believe that some benefits will be observable, and no matter how small, they will still be valuable for the goal of reaching entropy.

Expected Outcomes. The overall outcome of this aim is to further compress the data given. Compressing pieces that the large language model fails to catch or compress inefficiently are the main places where traditional compressors can prove useful.

Potential Pitfalls and Alternative Strategies. We expect that weighing which traditional compressors to use in our project will be a place of confusion. Each traditional model has its own

approach to compression, and as such, using one model over another may lead to a higher compression even if individually, the opposite is true.

Specific Aim #3: Prove the model is lossless and compare the final entropy rate with the theoretical limit as calculated with the appropriate formula from Shannon (1948).

The objective of this aim is to prove the effectiveness of the model. We will first calculate the final entropy rate that the model produces and comparing this with the theoretical limit as found with formulas from Shannon (1948) will tell us how far off our model is from entropy. We will then implement the final block of our project in order to prove that our model is truly lossless. Using a simple loop upon decompression to scan through a file and compare it to the original will give an accurate depiction of whether or not our model is lossless. Our rationale for finding how far off our model is from entropy is that it will tell us whether or not future work can be done to better the results found in this paper. Our rationale for seeing whether or not our model is lossless is that when AI is involved in a project, it is now working with probability, and uncertainty may lead to a loss of data. The goal is that the model does not lose anything upon compression, and so making sure of such is essential.

Justification & Summary of Preliminary Data. Arithmetic coding is a widely standardized technique in learned lossless compression papers today. It is a generally trusted method and as such should provide accurate and reliable results. Similarly, the methods described in Shannon (1948) are also common today and just as trusted. Thus, the formulas in Shannon should be able to accurately describe the entropy limit of a piece of data.

Expected Outcomes. The overall outcome of this aim is that any data that is compressed using the network is near or at the entropy limit. We also expect that all data that goes into the algorithm for compression will come back out exactly the same after decompression.

Potential Pitfalls and Alternative Strategies. We expect that the math described in Shannon (1948) will be difficult to apply to a wide variety of datasets. Unfortunately, it seems there isn't an alternative strategy as Shannon's formulas have been standardized.

Section III: Resources/Equipment

In this project, various pieces of software are being sourced from other sources. The large language model is likely to be sourced from LLaMA-2.0 (Touvron et al., 2023). The arithmetic encoder will be sourced from another paper due to being standardized. Each traditional compressor will be sourced from their patent or respective research paper. Besides software, hardware resources are also essential for this project's success. Two different specifications were initially being used: one computer containing an Intel i9-12900K (3.2 GHz), an RTX 3050, with 32 GB of RAM; the other containing an AMD Threadripper (4.0 GHz), an NVIDIA RTX A4500, and 64 GB of RAM. Later in the project, hardware was standardized to always run on the latter computer.

Section V: Ethical Considerations

Due to the nature of my project being purely digital, safety shouldn't be a large concern. This concern is further dampened by the fact that all experiments are designed to run locally from start to end, eliminating the risk of data being breached by others. One possible concern is that data to be compressed is lost in compression, possibly losing the user's information. To combat this concern, a block to check for loss of information is implemented, as outlined in specific aim #4.

Section VI: Timeline

Linked is a [Gantt chart](#) made in Google Sheets outlining the timeline of this project. The formatting of this Gantt chart was inspired by the work of Niranjana Nair, a Junior at the Massachusetts

Academy of Math and Science during the 2024-2025 school year, on his Independent STEM Research Project (Nair, 2025).

Section VIII: References

- Fu, H., Liang, F., Lei, B., Bian, N., zhang, Q., Akbari, M., Liang, J., & Tu, C. (2020). Improved Hybrid Layered Image Compression using Deep Learning and Traditional Codecs. *Signal Processing: Image Communication*, 82, 115774. <https://doi.org/10.1016/j.image.2019.115774>
- Bellard, F. (n.d.). *NNCP: Lossless Data Compression with Neural Networks*. Retrieved December 5, 2025, from <https://bellard.org/nncp/>
- Byron, K. (n.d.). *Byron Knoll—Cmix*. Retrieved December 7, 2025, from <https://www.byronknoll.com/cmixon.html>
- Goyal, M., Tatwawadi, K., Chandak, S., & Ochoa, I. (2018). *DeepZip: Lossless Data Compression using Recurrent Neural Networks* (No. arXiv:1811.08162). arXiv. <https://doi.org/10.48550/arXiv.1811.08162>
- Goyal, M., Tatwawadi, K., Chandak, S., & Ochoa, I. (2020). *DZip: Improved general-purpose lossless compression based on novel neural network modeling* (No. arXiv:1911.03572). arXiv. <https://doi.org/10.48550/arXiv.1911.03572>
- Mahoney, M. (2025, July 3). *Large Text Compression Benchmark*. Retrieved November 30, 2025, from <https://mattmahoney.net/dc/text.html>
- Mahoney, M. (2011, September 1). *About the Test Data*. Retrieved November 30, 2025, from <https://mattmahoney.net/dc/textdata.html>
- Mahoney, M. (n.d.). *Silesia Open Source Compression Benchmark*. Retrieved December 7, 2025, from <https://mattmahoney.net/dc/silesia.html>
- Mao, Y., Cui, Y., Kuo, T.-W., & Xue, C. J. (2022). *A Fast Transformer-based General-Purpose Lossless Compressor* (No. arXiv:2203.16114). arXiv. <https://doi.org/10.48550/arXiv.2203.16114>
- Moffat, A. (2019). Huffman Coding. *ACM Computing Surveys*, 52(4), 85:1-85:35. <https://doi.org/10.1145/3342555>

Nair, N. (2025) *Niranjan Nair - Gantt Chart* [Spreadsheet].

<https://docs.google.com/spreadsheets/d/1B2KZ0Vvpsw4j2iONyMWE402LQwTU35sVtnhOMxrNAKc/edit?gid=0#gid=0>

Qwen/Qwen3-8B-GGUF · Hugging face. (2025, December 31). Retrieved February 9, 2026, from

<https://huggingface.co/Qwen/Qwen3-8B-GGUF>

Schmidhuber, J., & Heil, S. (1996). Sequential neural text compression. *IEEE Transactions on Neural*

Networks, 7(1), 142–146. <https://doi.org/10.1109/72.478398>

Shannon, C. E. (1948). A mathematical theory of communication. *Bell System Technical Journal*, 27(3), 379–423; 27(4), 623–656.

Sun, H., Ma, H., Ling, F., Xie, H., Sun, Y., Yi, L., Yan, M., Zhong, C., Liu, X., & Wang, G. (2025). A survey and benchmark evaluation for neural-network-based lossless universal compressors toward multi-source data. *Frontiers of Computer Science*, 19(7), 197360. <https://doi.org/10.1007/s11704-024-40300-5>

Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., Rodriguez, A., Joulin, A., Grave, E., & Lample, G. (2023). *LLaMA: Open and Efficient Foundation Language Models* (No. arXiv:2302.13971). arXiv.

<https://doi.org/10.48550/arXiv.2302.13971>

Valmeekam, C. S. K., Narayanan, K., Kalathil, D., Chamberland, J.-F., & Shakkottai, S. (2023). *LLMZip: Lossless Text Compression using Large Language Models* (No. arXiv:2306.04050). arXiv.

<https://doi.org/10.48550/arXiv.2306.04050>

Yang, Y., Mandt, S., & Theis, L. (2023). *An Introduction to Neural Data Compression* (No.

arXiv:2202.06533). arXiv. <https://doi.org/10.48550/arXiv.2202.06533>